

Master Project Blueprint

Zero-State AI Resume & Portfolio Builder

Part 1: Product Requirement Document (PRD)

1.1 Executive Summary

The product is an instantaneous, zero-friction AI-powered Resume and Portfolio generator designed to be built and deployed within 24 hours. It eliminates traditional user authentication and database dependencies, acting as a stateless web application. Users interact with a conversational AI "Recruiter" that extracts high-impact metrics to dynamically generate a perfectly formatted, ATS-compliant PDF resume.

1.2 Target Audience & Use Case

- **Primary Audience:** Job seekers, software engineers, and professionals needing immediate, optimized resumes without signing up for expensive SaaS products.
- **Core Problem Solved:** Users typically write weak, non-metric-driven resumes. The AI fixes this by specifically prompting them for X-Y-Z metrics (Accomplished X, measured by Y, by doing Z).

1.3 Core User Flow

1. **Landing:** User enters the web app. No login screen. They immediately see a split-screen UI (Chat on left, Preview on right).
2. **Interview:** The AI prompts the user for their name and target role, then asks 3-5 specific questions about past projects.
3. **Compilation:** Once sufficient data is collected, the AI silently formulates a strictly structured JSON object.
4. **Delivery:** The frontend parses the JSON, immediately renders a PDF via React-PDF, and displays a download button. The session is complete. Data clears upon tab close.

1.4 Out of Scope (Strict)

To adhere to the 24-hour delivery constraint, the following are strictly prohibited in V1:

- User Login / Authentication (Clerk, Auth0, Supabase Auth).
- Database storage for user history or past resumes.
- Multi-column, graphic-heavy resume templates (these fail ATS parsers anyway).

Part 2: Technical Requirement Document (TRD)

2.1 Technology Stack

- **Frontend/Framework:** Next.js 14+ (App Router), React, TypeScript.
- **Styling & Motion:** Tailwind CSS, Framer Motion.
- **AI Provider:** Google Gemini API (Model: `gemini-2.5-flash` for lowest latency).
- **PDF Generation:** `@react-pdf/renderer` (Client-side vector generation).

2.2 Architecture & State Management

The application relies entirely on **React State** and **Stateless API Routes**.

- `messages[]`: Array of chat objects sent back and forth to Gemini.
- `resumeData{}`: A strongly typed JSON object populated only when the AI interview concludes.

2.3 API Configuration (Gemini Prompt Strategy)

The Next.js backend route must use the following configuration to ensure structural reliability:

System Instruction Trigger: The prompt must explicitly state: "Once you have gathered Name, Role, Skills, and 2 Projects, DO NOT converse further. Output exactly one valid JSON object and nothing else."

2.4 JSON Schema Contract

The AI must adhere to this exact output structure for the PDF renderer to succeed:

```
{
  "personalInfo": {
    "name": "string",
    "email": "string",
    "phone": "string"
  },
  "summary": "string",
  "skills": ["string", "string"],
  "projects": [
    {
      "name": "string",
      "techStack": ["string"],
      "bullets": ["string"]
    }
  ]
}
```

Part 3: Step-by-Step Implementation Guide

This section contains the highly detailed, micro-step execution plan to build the application from scratch.

Phase 1: Environment Setup & Installs

Step 1.1: Bootstrap Next.js

Open your terminal and run: `npx create-next-app@latest ai-resume-builder`

When prompted, select: TypeScript (Yes), ESLint (Yes), Tailwind CSS (Yes), `src/` directory (No), App Router (Yes).

Step 1.2: Install Dependencies

Navigate into the folder: `cd ai-resume-builder`

Install required packages: `npm install @react-pdf/renderer framer-motion @google/genai`

Step 1.3: Environment Variables

Create a `.env.local` file in the root directory.

Add your Gemini API Key: `GEMINI_API_KEY=your_actual_key_here`

Phase 2: Types & Shared Architecture

Step 2.1: Define Global Types

Create a file: `types/index.ts`

Add the exact TypeScript interfaces so the frontend and PDF builder align:

```
export interface Message {
  role: 'user' | 'model';
  content: string;
}

export interface Project {
  name: string;
  techStack: string[];
  bullets: string[];
}

export interface ResumeData {
  personalInfo: { name: string; email: string; phone: string; };
  summary: string;
  skills: string[];
  projects: Project[];
}
```

Phase 3: The API Route (Gemini Integration)

Step 3.1: Create the Route Handler

Create a file at: `app/api/chat/route.ts`

Step 3.2: Write the API Logic

Copy the following exact code. This handles the stateless memory by receiving the full array of messages.

```

import { GoogleGenAI } from '@google/genai';
import { NextResponse } from 'next/server';

const ai = new GoogleGenAI({ apiKey: process.env.GEMINI_API_KEY });

const SYSTEM_INSTRUCTION = `
You are an expert ATS Resume Recruiter.
Interview the user one question at a time to get their Name, Target Role, Skills, and
at least 2 Projects with X-Y-Z metrics.
Once you have this data, STOP conversing. Output ONLY a valid JSON object matching
this structure:
{"personalInfo": {"name": "", "email": "", "phone": ""}, "summary": "", "skills": [],
"projects": [{"name": "", "techStack": [], "bullets": []}]}
Do not include markdown formatting.
`;

export async function POST(req: Request) {
  const { messages } = await req.json();

  const formattedContents = messages.map((m: any) => ({
    role: m.role,
    parts: [{ text: m.content }]
  }));

  try {
    const response = await ai.models.generateContent({
      model: 'gemini-2.5-flash',
      contents: formattedContents,
      config: {
        systemInstruction: SYSTEM_INSTRUCTION,
        temperature: 0.2
      }
    });
  } catch (error) {
    return NextResponse.json({ error: "Failed to fetch" }, { status: 500 });
  }

  return NextResponse.json({ text: response.text });
}

```

Phase 4: The PDF Generator Component

Step 4.1: Create the Document Component

Create a file at: `components/ResumePDF.tsx`

Step 4.2: Write the React-PDF Layout

This component strictly avoids multi-column layouts to ensure perfect ATS parsing.

```
import { Document, Page, Text, View, StyleSheet } from '@react-pdf/renderer';
import { ResumeData } from '../types';

const styles = StyleSheet.create({
  page: { padding: 40, fontFamily: 'Helvetica' },
  header: { fontSize: 24, marginBottom: 10, textAlign: 'center' },
  contact: { fontSize: 10, textAlign: 'center', marginBottom: 20 },
  sectionTitle: { fontSize: 14, borderBottom: '1px solid black', marginBottom: 10,
marginTop: 15 },
  text: { fontSize: 10, marginBottom: 5 },
  bullet: { fontSize: 10, marginLeft: 10, marginBottom: 4 }
});

export const ResumePDF = ({ data }: { data: ResumeData }) => (

  {data.personalInfo.name}
  {data.personalInfo.email} | {data.personalInfo.phone}

  Summary
  {data.summary}

  Skills
  {data.skills.join(', ')}

  Projects
  {data.projects.map((proj, i) => (

    {proj.name} ({proj.techStack.join(', ')}
    {proj.bullets.map((b, idx) => (
      • {b}
    ))}

  ))}

);
```

Phase 5: The Main Client UI

Step 5.1: Overwrite `app/page.tsx`

This file handles the chat UI, API calling, JSON parsing, and rendering the PDF download button.

Step 5.2: Implement Client Logic

```

'use client';
import { useState } from 'react';
import { Message, ResumeData } from '../types';
import { ResumePDF } from '../components/ResumePDF';
import { PDFDownloadLink } from '@react-pdf/renderer';

export default function Home() {
  const [messages, setMessages] = useState([
    { role: 'model', content: 'Hello! What is your name and target job role?' }
  ]);
  const [input, setInput] = useState('');
  const [resumeData, setResumeData] = useState(null);

  const sendMessage = async () => {
    if (!input) return;
    const newMsgs: Message[] = [...messages, { role: 'user', content: input }];
    setMessages(newMsgs);
    setInput('');

    const res = await fetch('/api/chat', {
      method: 'POST',
      body: JSON.stringify({ messages: newMsgs })
    });
    const data = await res.json();
    const text = data.text.trim();

    // Micro-step logic: Detect JSON payload
    if (text.startsWith('{') && text.endsWith('}')) {
      try {
        setResumeData(JSON.parse(text));
        setMessages([...newMsgs, { role: 'model', content: 'Resume built! Download it now.' }]);
      } catch (e) {
        setMessages([...newMsgs, { role: 'model', content: 'Error parsing JSON data.' }]);
      }
    } else {
      setMessages([...newMsgs, { role: 'model', content: text }]);
    }
  };

  return (

    { /* LEFT PANEL */ }
  )
}

```

AI Interview

```
{messages.map((m, i) => (  
  
    {m.content}  
  
    )))  
{input} setInput(e.target.value)}  
    style={{ width: '80%', padding: '10px', color: 'black' }}  
/>  
    Send
```

```
{/* RIGHT PANEL */}
```

```
{resumeData ? (
  } fileName="Resume.pdf">
  ({({ loading }) => loading ? 'Generating...' :
Download ATS PDF})
) : (
```

 $\left. \vphantom{\int} \right\}$

```

    );
}

```

Step 5.3: Run the Application

Run `npm run dev` in your terminal.

Open `http://localhost:3000` in your browser.

Engage with the chat bot until it extracts enough information to dynamically populate the right panel with your perfect PDF!